ОТЧЁТ

Практическая работа № 3, раздел 1 «Процессы, сигналы и управление заданиями Linux»

Выполнил: Калинин Илья

Учётная запись: user1

Сервер: eltex-practice2-pg1-v20

Модуль 1. Практика 3. Элтекс. 2026 год.

Цель и условия работы

Цель: изучить процессы и потоки Linux, мониторинг в top, управление заданиями оболочки, изменение приоритета NICE, доставку и перехват сигналов, а также особенности процессов в состоянии zombie. Работа выполнялась в терминале Linux на удалённом сервере под пользователем user1.

Отчёт составлен по методичке и сохранённым материалам практики. Там, где в доступных скриншотах нет однозначного подтверждения конкретного шага, это прямо указано; значения и результаты не реконструируются как якобы полученные.

Процессы и потоки (задания 1–2)

su - user1 ps -eLf

Команда ps -eLf выводит сведения о потоках (LWP) и числе потоков NLWP. Для подсчёта многопоточных процессов необходимо сгруппировать строки по PID либо выбрать уникальные PID с NLWP больше единицы. Численный итог подсчёта в сохранившихся данных однозначно не установлен.

Мониторинг top (задание 3)

В top исследуются три показателя памяти: VIRT — виртуальное адресное пространство процесса, RES — находящаяся в физической памяти часть, SHR — оценка разделяемой памяти. Сортировка по каждому полю по убыванию позволяет увидеть процессы с наибольшими соответствующими показателями. USER показывает эффективного пользователя процесса, RUSER — реального пользователя; эти значения могут отличаться, например, у программ с изменёнными правами запуска.

Методичка требует три отдельных скриншота сортировки VIRT, RES и SHR и размещение RUSER сразу после USER. Полный комплект таких подтверждений среди доступных материалов не установлен.

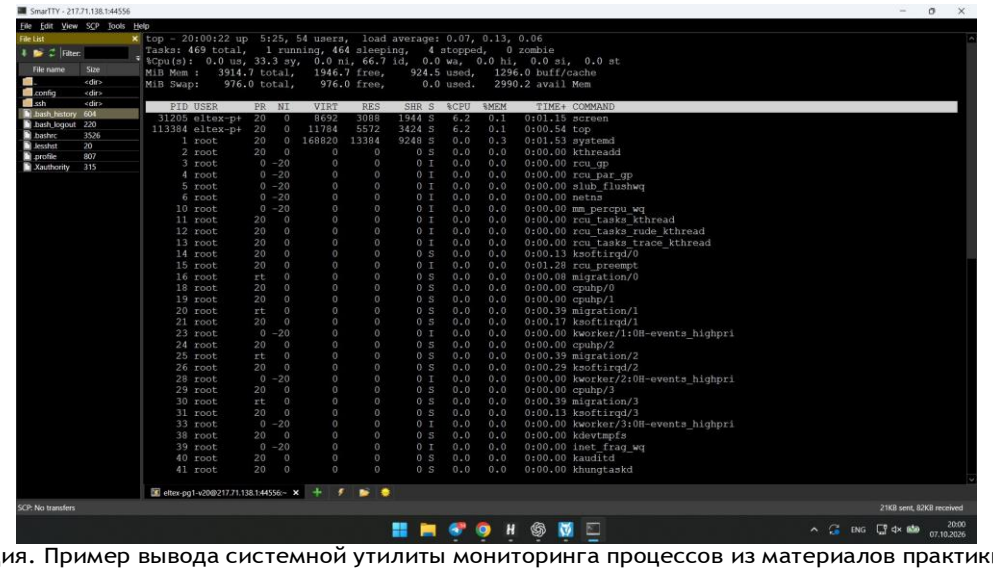


Иллюстрация. Пример вывода системной утилиты мониторинга процессов из материалов практики.

passwd и сигналы (задания 4–5)

passwd

ps -ef | grep passwd kill -15 PID # SIGTERM kill -2 PID # SIGINT
kill -3 PID # SIGQUIT kill -9 PID # SIGKILL
Для каждого испытания сигналов нужен работающий экземпляр passwd; после завершения процесса повторная отправка по тому же PID не сработает. SIGTERM запрашивает завершение, SIGINT соответствует прерыванию, SIGQUIT обычно завершает процесс с возможностью дампа, SIGKILL принудительно уничтожает процесс и не перехватывается. По имеющимся материалам нельзя утверждать, что все четыре сигнала были отправлены отдельным процессам passwd и зафиксированы.

Задания оболочки и NICE (задания 6–12)
vim ~/file_task3.txt Ctrl+Z
sleep 600

Ctrl+Z; jobs; bg %2 renice -n 10 -p PID
ps -o pid,ni,stat,cmd -p PID fg %1; kill -TERM %2; jobs
Ctrl+Z приостанавливает задание; jobs показывает задания оболочки; bg переводит приостановленное задание в фон, fg возвращает его на передний план. NICE=10 означает более низкий приоритет планирования относительно NICE=0. Номер задания и PID необходимо определять по фактическому выводу jobs/ps: приведённые %1 и %2 — примеры, не зафиксированные идентификаторы.



Иллюстрация. Фрагмент терминальной работы с процессами и заданиями.

Перехват сигналов (задание 13)

trap 'echo Меня голыми руками не возьмёшь!' INT QUIT

Встроенная команда trap устанавливает обработчики SIGINT и SIGQUIT в текущей оболочке. В практической работе дополнительно создавался сценарий trap_test.sh для экспериментов с обработкой сигналов. Сигнал SIGKILL перехватить нельзя.



Иллюстрация. Фрагмент экспериментов с сигналами в терминале.

Zombie-процессы (задания 14–15)

(sleep 1 & exec /bin/sleep 30) &

ps -eo pid,ppid,stat,cmd | grep -E " Z|defunct"

В ходе практики дополнительно использовался файл zombie.py, который создаёт дочерний процесс с помощью os.fork(). Дочерний процесс завершается вызовом os._exit(0), а родитель остаётся активным некоторое время. В наблюдении был зафиксирован дочерний PID 10958 со статусом Z и родительский PID 10957 со статусом
S. После завершения родителя оба процесса исчезли из списка.

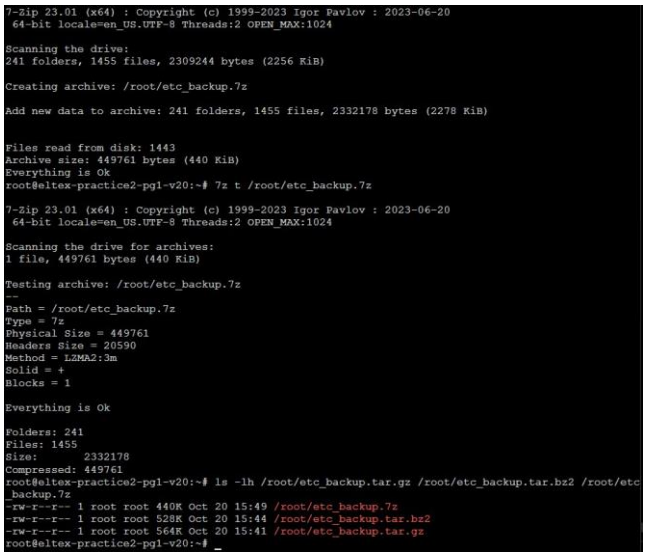


Иллюстрация. Фрагмент диагностики процессов в терминале.

Особенности zombie-процессов (объяснение)

Zombie — процесс, выполнение которого уже завершилось, но запись о нём ещё сохраняется в таблице процессов.
Его состояние обозначается буквой Z в столбце STAT утилиты ps.

Zombie не исполняет пользовательский код и не расходует процессорное время.

Запись сохраняется, чтобы родитель мог получить код завершения дочернего процесса.
Обычно родитель вызывает wait() или waitpid() и удаляет такую запись.

Пока родитель не получил статус завершения, запись может оставаться.

SIGTERM или SIGKILL не могут завершить zombie повторно: он уже завершился.

При завершении родителя осиротевшего потомка усыновляет соответствующий процесс-сборщик, который может получить его статус.
Большое количество zombie указывает на возможные ошибки управления дочерними процессами.
При диагностике полезно выводить одновременно PID, PPID и STAT, чтобы определить родительский процесс и состояние потомка.

Вывод

В работе рассмотрены процессы и потоки Linux, поля мониторинга top, сигналы, управление заданиями и приоритет NICE. Практический эксперимент с zombie подтвердил, что завершившийся дочерний процесс может временно сохраняться в состоянии Z, пока жив его родитель. Для строгого соответствия требованиям преподавателя необходимо дополнить отчёт недостающими точными подтверждениями сортировок top и отдельных действий с passwd, если такие скриншоты имеются.